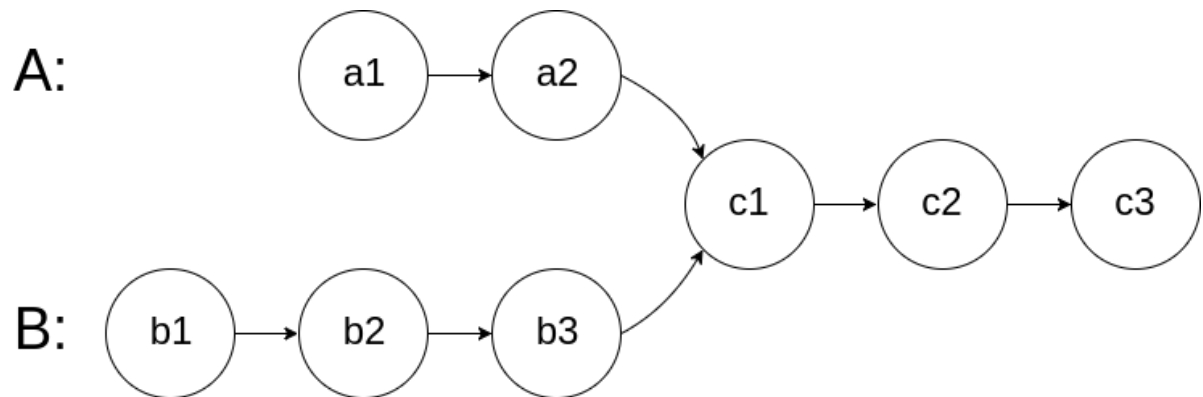


160.相交链表

[哈希表](#) | [链表](#) | [双指针](#)

给你两个单链表的头节点 `headA` 和 `headB`，请你找出并返回两个单链表相交的起始节点。如果两个链表不存在相交节点，返回 `null`。

图示两个链表在节点 `c1` 开始相交：



题目数据 **保证** 整个链式结构中不存在环。

注意，函数返回结果后，链表必须 **保持其原始结构**。

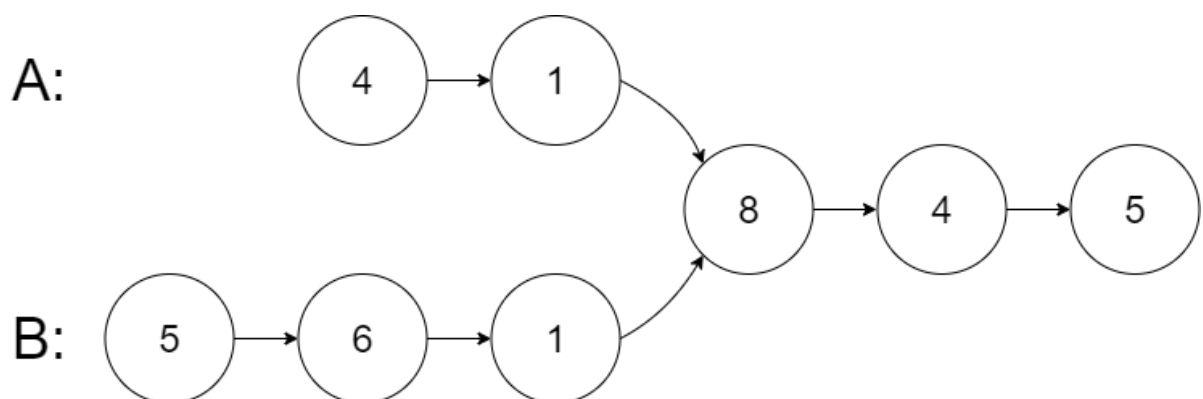
自定义评测：

评测系统 的输入如下（你设计的程序 **不适用** 此输入）：

- `intersectVal` - 相交的起始节点的值。如果不存在相交节点，这一值为 `0`
- `listA` - 第一个链表
- `listB` - 第二个链表
- `skipA` - 在 `listA` 中（从头节点开始）跳到交叉节点的节点数
- `skipB` - 在 `listB` 中（从头节点开始）跳到交叉节点的节点数

评测系统将根据这些输入创建链式数据结构，并将两个头节点 `headA` 和 `headB` 传递给你的程序。如果程序能够正确返回相交节点，那么你的解决方案将被 **视作正确答案**。

示例 1：

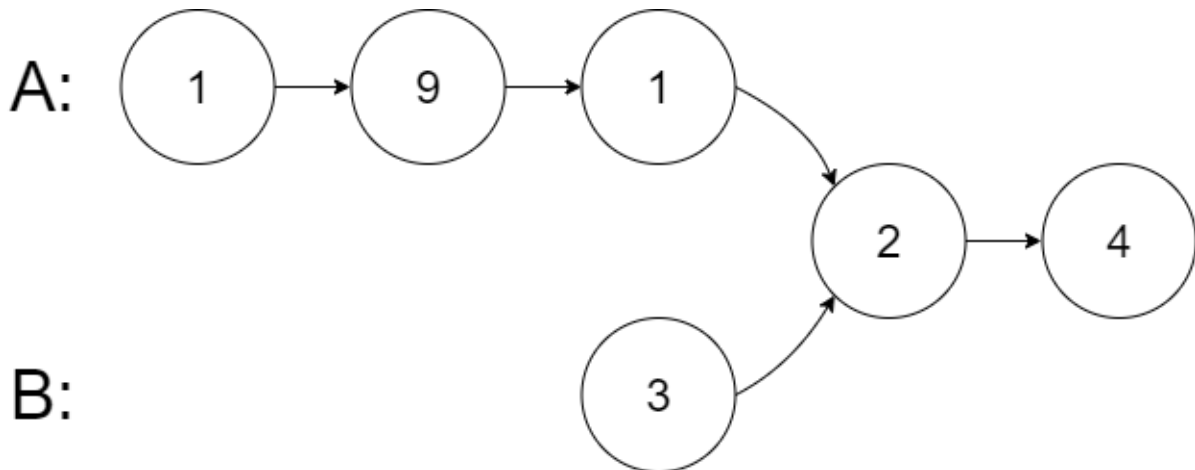


```

1  输入: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5], skipA = 2,
   skipB = 3
2  输出: Intersected at '8'
3  解释: 相交节点的值为 8 （注意，如果两个链表相交则不能为 0）。
4  从各自的表头开始算起，链表 A 为 [4,1,8,4,5]，链表 B 为 [5,6,1,8,4,5]。
5  在 A 中，相交节点前有 2 个节点；在 B 中，相交节点前有 3 个节点。
6  - 请注意相交节点的值不为 1，因为在链表 A 和链表 B 之中值为 1 的节点（A 中第二个节点和 B
   中第三个节点）是不同的节点。换句话说，它们在内存中指向两个不同的位置，而链表 A 和链表 B 中值
   为 8 的节点（A 中第三个节点，B 中第四个节点）在内存中指向相同的位置。

```

示例 2:

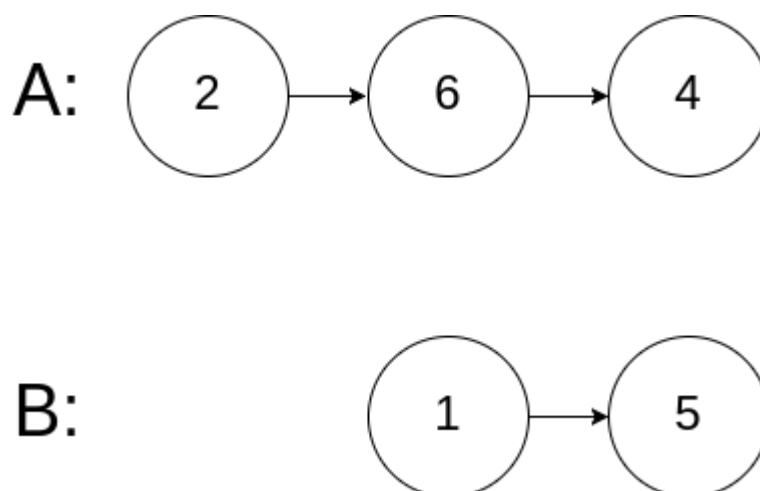


```

1  输入: intersectVal = 2, listA = [1,9,1,2,4], listB = [3,2,4], skipA = 3, skipB
   = 1
2  输出: Intersected at '2'
3  解释: 相交节点的值为 2 （注意，如果两个链表相交则不能为 0）。
4  从各自的表头开始算起，链表 A 为 [1,9,1,2,4]，链表 B 为 [3,2,4]。
5  在 A 中，相交节点前有 3 个节点；在 B 中，相交节点前有 1 个节点。

```

示例 3:



```

1  输入: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
2  输出: No intersection
3  解释: 从各自的表头开始算起，链表 A 为 [2,6,4]，链表 B 为 [1,5]。
4  由于这两个链表不相交，所以 intersectVal 必须为 0，而 skipA 和 skipB 可以是任意值。
5  这两个链表不相交，因此返回 null。

```

提示:

- `listA` 中节点数目为 `m`
- `listB` 中节点数目为 `n`
- `1 <= m, n <= 3 * 104`
- `1 <= Node.val <= 105`
- `0 <= skipA <= m`
- `0 <= skipB <= n`
- 如果 `listA` 和 `listB` 没有交点, `intersectVal` 为 `0`
- 如果 `listA` 和 `listB` 有交点, `intersectVal == listA[skipA] == listB[skipB]`

进阶: 你能否设计一个时间复杂度 `O(m + n)`、仅用 `O(1)` 内存的解决方案?

[Discussion](#) | [Solution](#)

2. 解决方法

LeetCode 160: 相交链表 (详细解析)

1. 问题理解

目标: 找出两个单链表的第一个相交节点。

输入:

- 两个单链表的头节点 `headA` 和 `headB`

输出:

- 如果两个链表相交, 返回相交的第一个节点
- 如果两个链表不相交, 返回 `null`

相交定义:

- 两个链表有一个公共节点, 且从该节点开始直到链表结束, 路径完全相同
- 这意味着两个链表在相交节点后共享完全相同的节点 (不仅仅是值相同, 而是节点引用相同)

约束条件:

- 不允许修改链表结构
- 时间复杂度应为 `O(m+n)`, 空间复杂度应为 `O(1)`
- 链表中节点数量范围为 `[0, 104]`

2. 普通人的思路 (及遇到的困难)

思路一: 使用哈希集合

想法: 把第一个链表的所有节点存入哈希集合, 然后检查第二个链表的每个节点是否在集合中。

实现步骤:

1. 创建一个哈希集合 `set`

2. 遍历链表 A，将每个节点加入 `set`
3. 遍历链表 B，对于每个节点，检查它是否在 `set` 中
4. 如果在，则该节点是相交点，返回它
5. 如果遍历完 B 也没找到，则返回 `null`

复杂度：

- 时间复杂度： $O(m+n)$
- 空间复杂度： $O(m)$ ，其中 m 是链表 A 的长度

缺点： 需要额外的 $O(m)$ 空间，不满足题目的空间 $O(1)$ 要求。

思路二：测量链表长度

想法： 计算两个链表的长度差，让较长的链表先走差值步数，然后同时遍历两个链表寻找相同节点。

实现步骤：

1. 计算链表 A 和链表 B 的长度
2. 计算两链表的长度差 `diff`
3. 让较长的链表先走 `diff` 步
4. 然后两个链表同时前进，比较每一对节点是否相同
5. 第一个相同的节点就是相交点

复杂度：

- 时间复杂度： $O(m+n)$
- 空间复杂度： $O(1)$

缺点： 实现较为繁琐，需要多次遍历链表。

3. 更优雅的方法（双指针技巧）

核心思想： 使用两个指针 `pA` 和 `pB`，让它们分别从链表 A 和链表 B 的头部开始遍历。当一个指针到达链表末尾时，将它重定向到另一个链表的头部，继续遍历。

为什么这样有效？

- 假设链表 A 长度为 a ，链表 B 长度为 b ，相交部分长度为 c 。
- 不相交的部分长度分别为： $a-c$ 和 $b-c$ 。
- 指针 `pA` 的路径：遍历链表 A(a 步) + 遍历链表 B(b 步) = $a+b$ 步
- 指针 `pB` 的路径：遍历链表 B(b 步) + 遍历链表 A(a 步) = $b+a$ 步
- 两个指针走过的总路径长度相同，都是 $a+b$ 步
- 如果有相交点，两个指针必然会在相交点相遇
- 如果没有相交点，两个指针最终都会到达 `null`

图解：

```
1 链表A: a1 → a2 → c1 → c2 → c3
2
3 链表B: b1 → b2
4
```

- 指针 `pA` 的路径: $a1 \rightarrow a2 \rightarrow c1 \rightarrow c2 \rightarrow c3 \rightarrow b1 \rightarrow b2 \rightarrow c1 \dots$
- 指针 `pB` 的路径: $b1 \rightarrow b2 \rightarrow c1 \rightarrow c2 \rightarrow c3 \rightarrow a1 \rightarrow a2 \rightarrow c1 \dots$
- 两个指针会在第一个相交节点 `c1` 处相遇

4. 示例演示（双指针法）

以示例 1 为例：

```
1 链表A: 4 → 1 → 8 → 4 → 5
2
3 链表B: 5 → 6 → 1
4
```

1. `pA` 指向链表 A 的头节点(4), `pB` 指向链表 B 的头节点(5)
2. `pA` 移动: $4 \rightarrow 1 \rightarrow 8 \rightarrow 4 \rightarrow 5 \rightarrow \text{null}$, 重置到链表 B 头节点: 5
3. `pB` 移动: $5 \rightarrow 6 \rightarrow 1 \rightarrow 8 \rightarrow 4 \rightarrow 5 \rightarrow \text{null}$, 重置到链表 A 头节点: 4
4. `pA` 继续: $5 \rightarrow 6 \rightarrow 1 \rightarrow 8$
5. `pB` 继续: $4 \rightarrow 1 \rightarrow 8$
6. 此时 `pA` 和 `pB` 都指向节点 8, 这就是相交点

5. 代码实现

```
1  /**
2   * Definition for singly-linked list.
3   * public class ListNode {
4   *     int val;
5   *     ListNode next;
6   *     ListNode(int x) {
7   *         val = x;
8   *         next = null;
9   *     }
10  * }
11  */
12 public class Solution {
13     /**
14      * 找出两个单链表的相交节点
15      * 方法: 双指针技巧
16      * @param headA 第一个链表的头节点
17      * @param headB 第二个链表的头节点
18      * @return 相交节点, 如果不存在则返回 null
19      */
20     public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
21         // 如果任一链表为空, 不可能相交
22         if (headA == null || headB == null) {
23             return null;
```

```

24     }
25
26     // 初始化两个指针
27     ListNode pA = headA;
28     ListNode pB = headB;
29
30     // 当 pA 和 pB 不相等时，继续移动
31     while (pA != pB) {
32         // 如果 pA 到达链表 A 的末尾，将其重定向到链表 B 的头部
33         // 否则，pA 向后移动
34         pA = (pA == null) ? headB : pA.next;
35
36         // 如果 pB 到达链表 B 的末尾，将其重定向到链表 A 的头部
37         // 否则，pB 向后移动
38         pB = (pB == null) ? headA : pB.next;
39     }
40
41     // 此时 pA 和 pB 要么都指向相交节点，要么都是 null（如果不相交）
42     return pA;
43 }
44 }
45

```

6. 代码解析

初始化：

- 首先检查两个链表是否为空，如果任一链表为空，不可能有相交点。
- 初始化两个指针 `pA` 和 `pB`，分别指向链表 A 和链表 B 的头节点。

主循环逻辑：

- 当 `pA` 和 `pB` 不相等时，继续循环。
- 如果 `pA` 到达链表 A 的末尾（即 `pA == null`），将其重定向到链表 B 的头部；否则，`pA` 向后移动一步。
- 同样，如果 `pB` 到达链表 B 的末尾，将其重定向到链表 A 的头部；否则，`pB` 向后移动一步。

循环终止条件：

- 当 `pA == pB` 时，循环终止。
- 此时，`pA` 和 `pB` 要么都指向相交节点，要么都是 `null`（如果两个链表不相交）。

返回值：

- 返回 `pA`（或 `pB`，它们相等）作为相交节点，或者 `null` 表示没有相交。

7. 复杂度分析

时间复杂度： $O(m + n)$

- 在最坏情况下，每个指针都需要遍历两个链表各一次，总路径长度为 $m+n$
- 如果有相交点，两个指针可能提前相遇，实际复杂度可能小于 $O(m+n)$

空间复杂度： $O(1)$

- 只使用了两个指针变量 `pA` 和 `pB`，不依赖于输入规模的额外空间

8. 如何在面试中讲解

1. 确认问题理解：

- "我需要找到两个单链表的第一个相交节点，相交意味着从某一点开始两个链表共享相同的节点序列直到末尾。"
- "如果不相交，应该返回 null，并且我需要满足 $O(m+n)$ 的时间复杂度和 $O(1)$ 的空间复杂度。"

2. 提出并比较可能的解法：

- "最直接的方法是使用哈希集合存储第一个链表的所有节点，然后遍历第二个链表检查交点。但这需要 $O(m)$ 的额外空间。"
- "另一个方法是计算两个链表的长度差，让较长的链表先走差值步数，然后同时遍历比较。这满足空间要求，但稍显繁琐。"
- "更优雅的方法是使用双指针技巧，这是我打算实现的方法。"

3. 解释双指针方法的核心思想：

- "用两个指针 `pA` 和 `pB` 分别从两个链表头部开始遍历。"
- "当一个指针到达链表末尾时，将它重定向到另一个链表的头部，继续遍历。"
- "如果链表相交，两个指针最终会在相交点相遇；如果不相交，它们最终都会变为 null。"

4. 证明算法正确性：

- "假设链表 A 长 a ，链表 B 长 b ，相交部分长 c 。"
- "指针 `pA` 走完链表 A 后转向链表 B，总路径长为 $a+b$ ；指针 `pB` 走完链表 B 后转向链表 A，总路径也是 $b+a$ 。"
- "由于总路径长相同，如果有相交点，两个指针必定会在相交点相遇；如果没有相交点，它们最终都会到达 null。"

5. 强调算法效率：

- "这个算法的时间复杂度是 $O(m+n)$ ，因为每个指针最多遍历两个链表一次。"
- "空间复杂度是 $O(1)$ ，只使用了两个指针变量，不需要额外空间。"

通过双指针技巧，我们找到了一个既满足效率要求，又实现简洁的解决方案。

找到具有 1 个许可证类型的类似代码